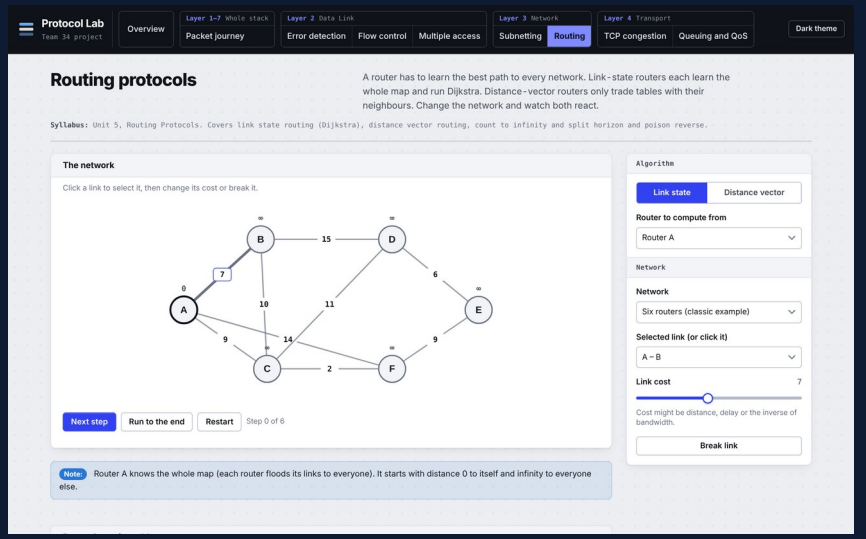
Protocol Lab: An Interactive Computer Networks Simulator

Watch network protocols work, then break them.



Savir Ghai · 24BCT0309

Team F1_Team34 · BCSE308P Computer Networks Lab

Faculty: Prof. Sreethar S

Protocols are taught as pictures

A diagram shows one case

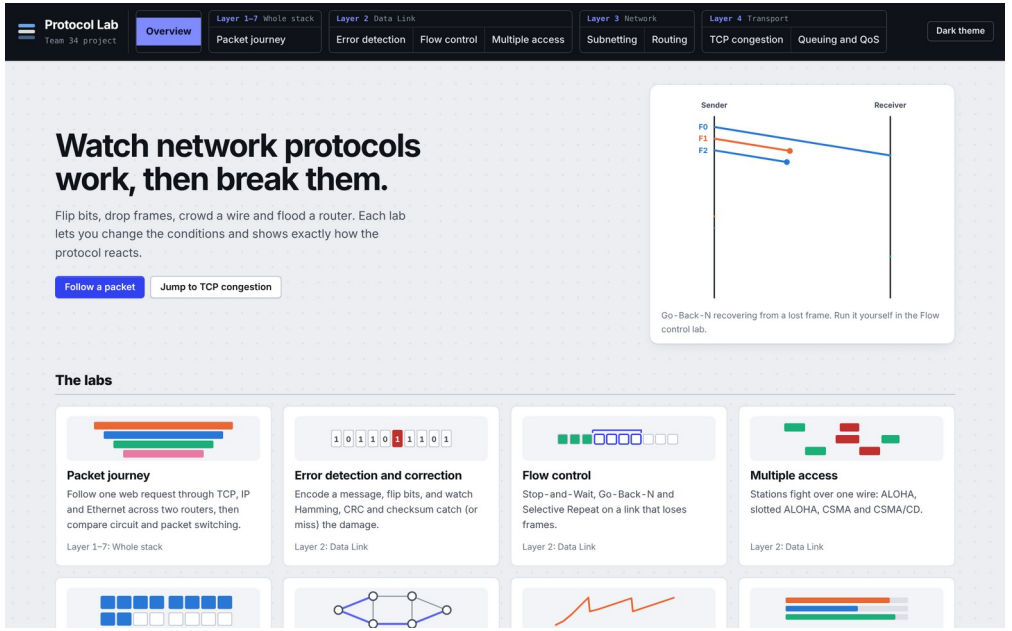
What happens to Go-Back-N when loss goes from 2% to 20%? A textbook figure cannot say.

Real simulators are heavy

ns-3 and Mininet need installation, scripting and time before a first result.

Packet Tracer was not usable here

And it hides protocol internals such as the sender window.



Existing approaches and the gap they leave

Approach	Strength	Gap
Textbooks and slides	Reliable explanations	Fixed examples; cannot be re-run with other parameters
ns-3 (Henderson 2008)	Accurate, standard in research	Install, C++ or Python scripting, slow to try one idea
Mininet (Lantz 2010)	Runs real network software	Linux and command line; not a view of protocol state
Cisco Packet Tracer	Topology and CLI practice	Not usable in this project; internals hidden

The gap

Nothing lightweight shows the baseline and the improved mechanism side by side, on identical input, with results you can trust and repeat.

The novelty is the combination, not a new protocol

Zero install

One HTML page. No framework, no build, no server, works offline.

Side by side

Baseline and improvement run on identical seeded input.

Verified

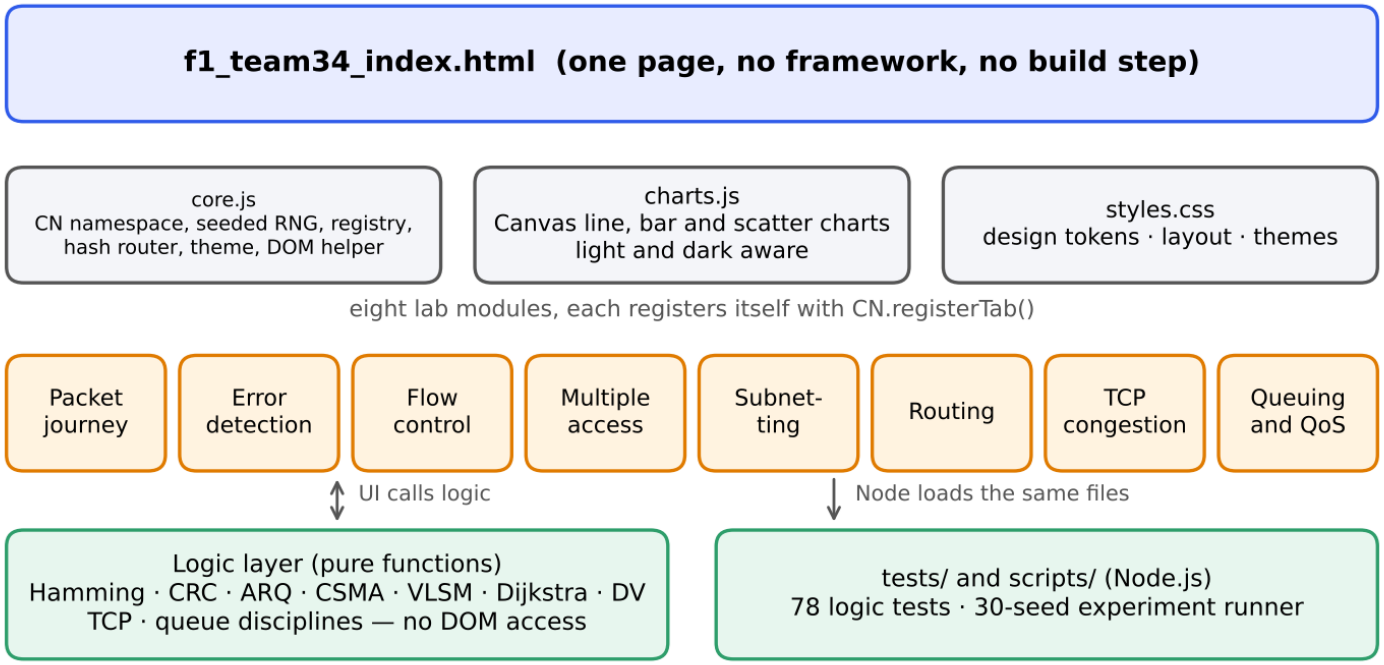
78 automated tests, including four known-answer checks.

Reproducible

Fixed seeds and a 30-seed runner give mean ± SD tables.

The mechanisms themselves (Selective Repeat, Reno, WFQ, split horizon) are classical. The report says so plainly.

Architecture: logic apart from the screen



One page, eight modules

Each lab registers itself with a tab registry; a hash router shows it.

Pure logic functions

A guard exits before any DOM code when run in Node, so tests and experiments load the same files.

Seeded random numbers

Same seed, same result, every time.

Implementation setup

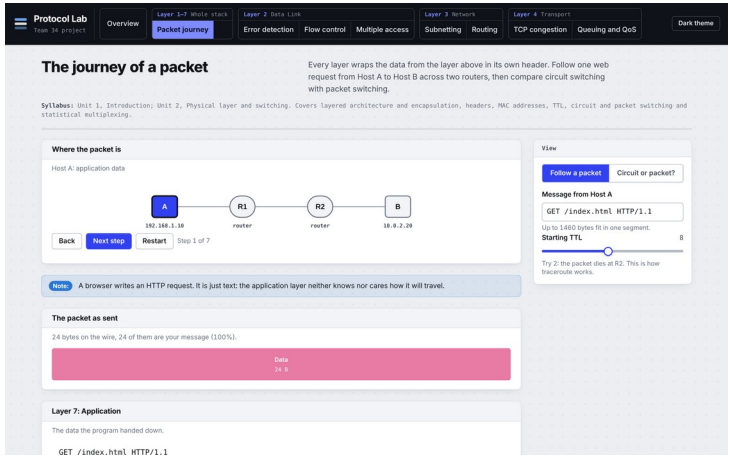
≈5,900
lines of JavaScript, CSS and tests

0
dependencies, frameworks or build steps

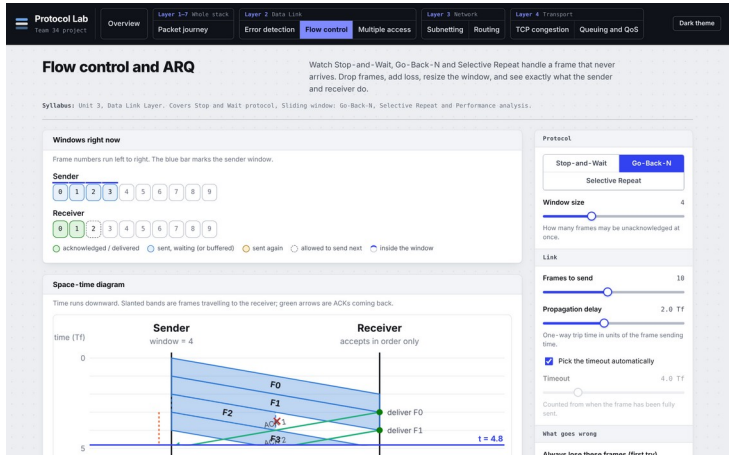
8
labs covering Units 1 to 6

Item	Detail
Languages	HTML5, CSS3, JavaScript (ES2020)
Run time	Any modern browser; Node.js v22 for tests and experiments
Screenshots	Chromium 153 headless (Playwright)
Machine for runs	Linux, 2 cores, 7 GB; all 30-seed runs finish in about 0.5 s
Repository	github.com/savir18/CN, files named f1_team34_*

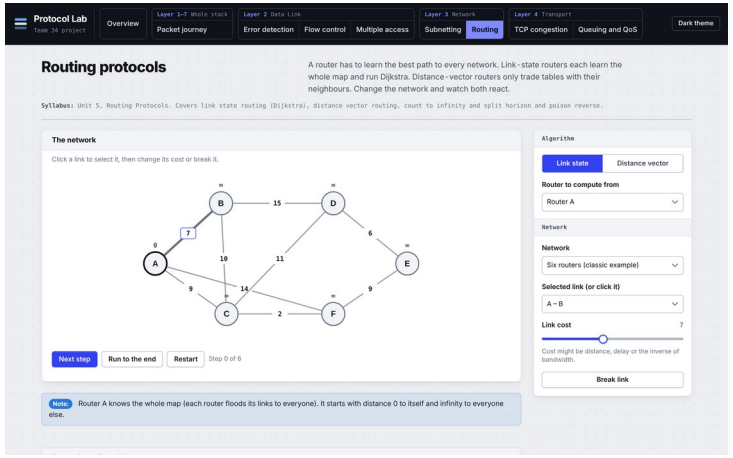
Eight labs, one per part of the syllabus



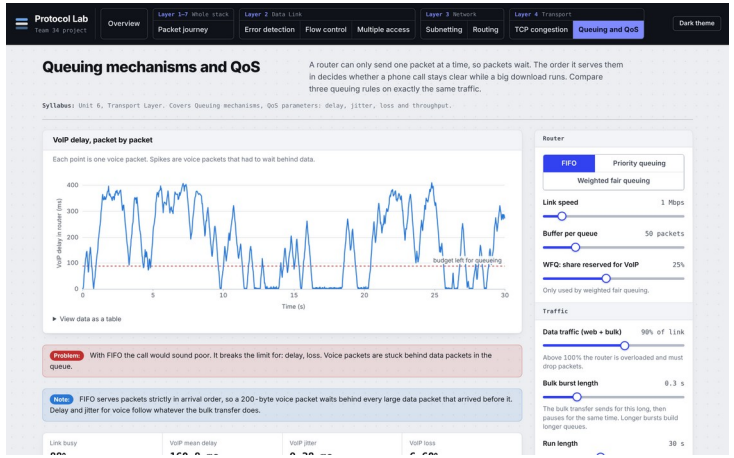
Packet journey (Units 1–2)



Flow control (Unit 3)



Routing (Unit 5)



Queuing and QoS (Unit 6)

Four comparisons, three scenarios each, 30 seeds

Lab	Baseline → proposed	Scenarios S1, S2, S3	Metrics
Routing	Plain distance vector → split horizon, poison reverse	Chains of 3 to 8 routers	Rounds until stable
Flow control	Go-Back-N → Selective Repeat	Frame loss 2%, 10%, 20%	Link efficiency; retransmissions per frame
TCP	Tahoe → Reno	Loss per round 2%, 5%, 10%	Segments sent in 100 RTTs
Queuing	FIFO → priority, WFQ	Data load 50%, 80%, 95%	Voice p95 delay; voice loss

Identical conditions for both mechanisms in every run. Mean ± sample SD of 30 seeds. Routing is deterministic, so one run per cell.

Routing: split horizon ends the 31-round loop

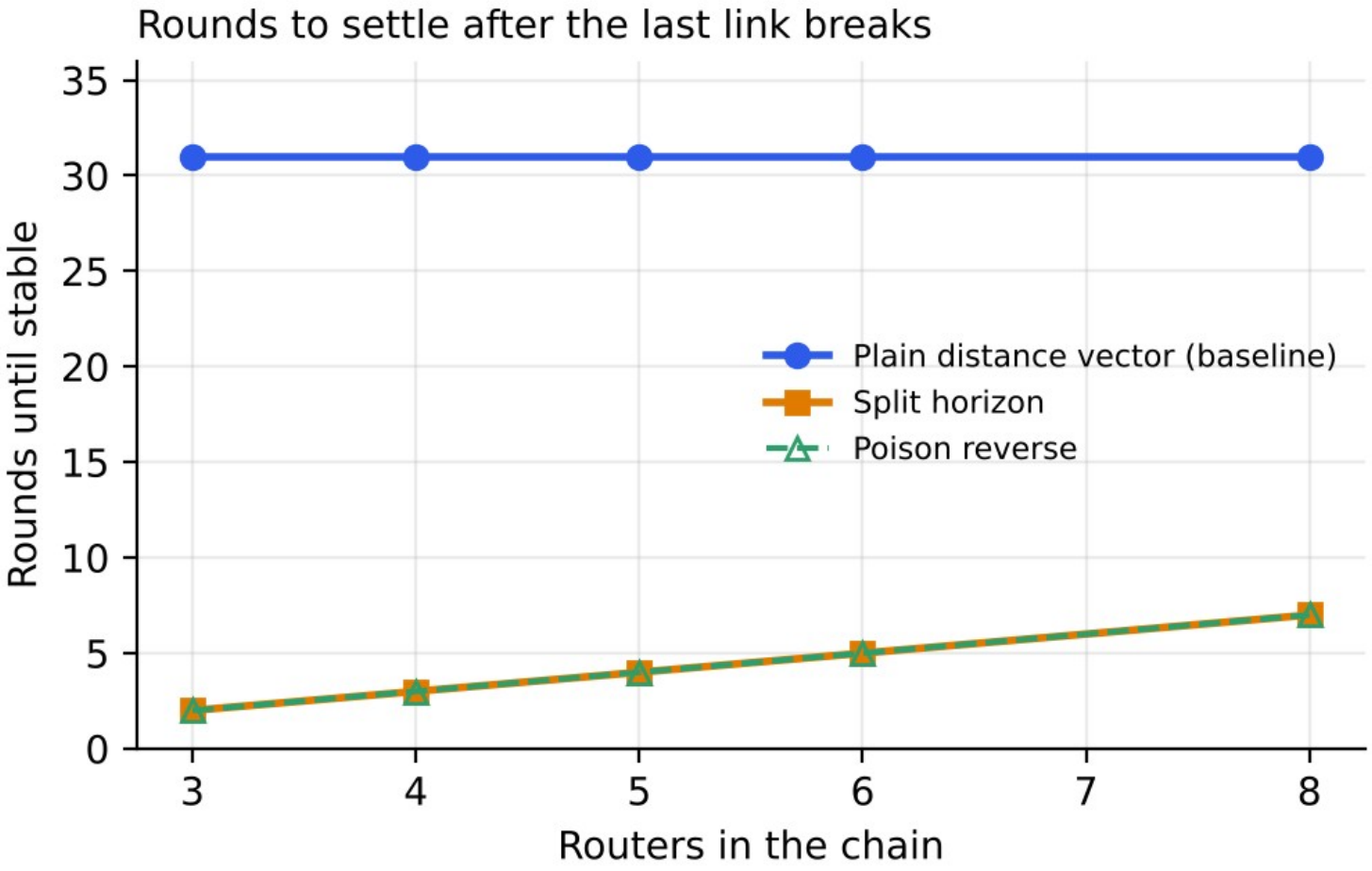
31

rounds with plain distance vector, in every chain

2 to 7

rounds with split horizon or poison reverse

Last link of a chain breaks. News of the loss travels one hop per round with split horizon; without it, two routers keep feeding each other the dead route until cost reaches infinity (32).



Flow control: Selective Repeat wins as loss grows

0.405 → 0.681

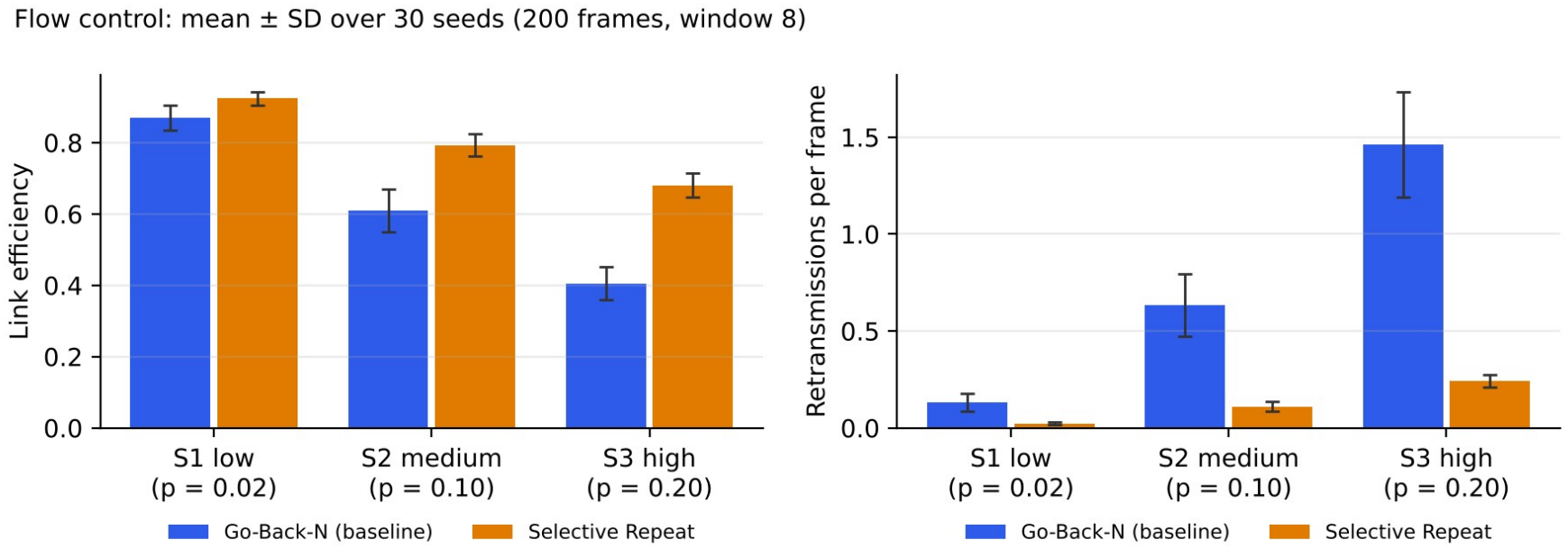
efficiency at 20% loss (GBN → SR)

1.46 → 0.25

retransmissions per frame at 20% loss

+6% → +68%

efficiency gain from S1 to S3



TCP: Reno halves its window, Tahoe restarts

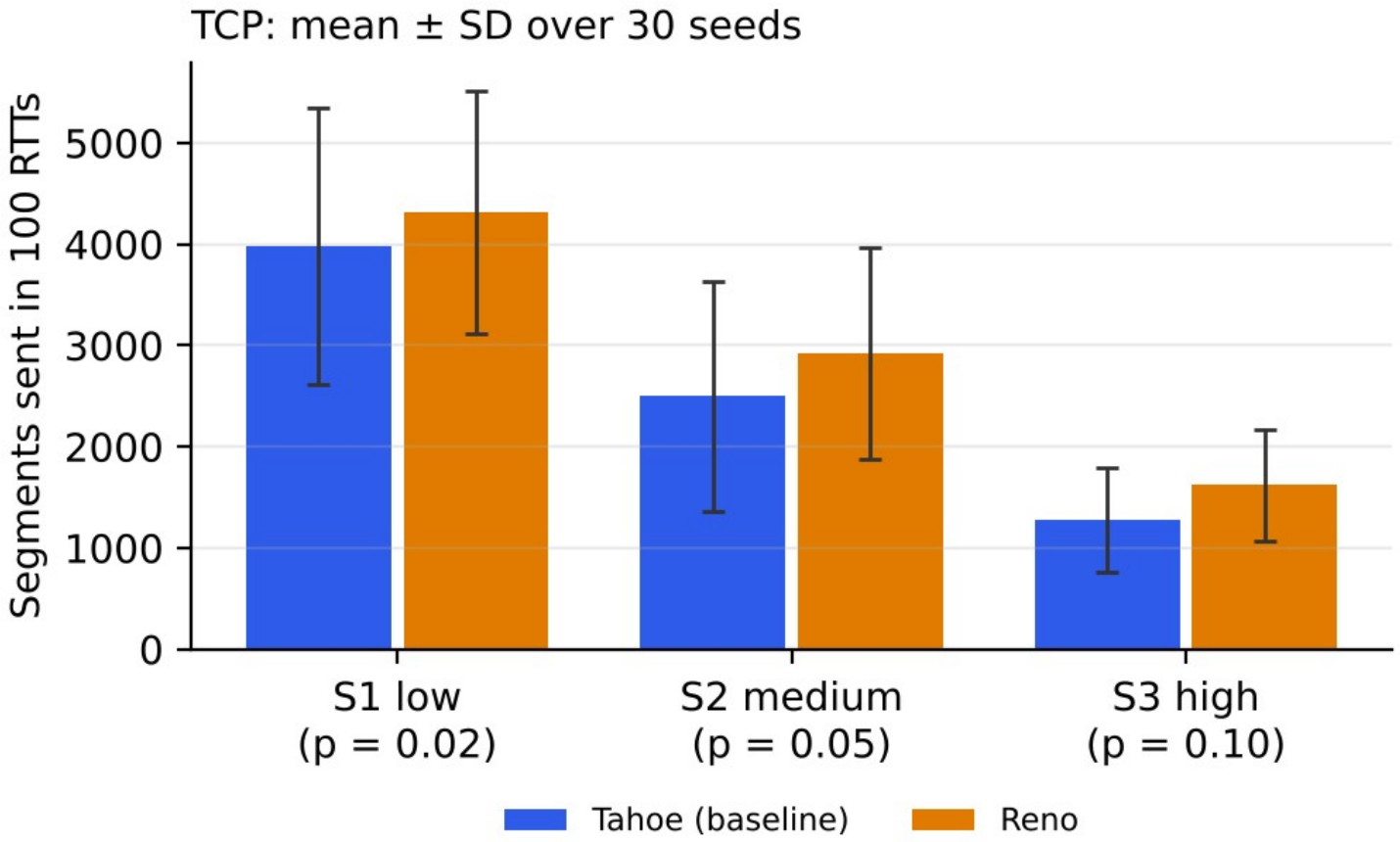
+8% to +27%

more segments for Reno than Tahoe

On three duplicate ACKs Reno halves its window; Tahoe falls back to 1. On a timeout both restart from 1.

Read with care

The model draws one loss event per round, whatever the window. The size of the gain is a property of the model. The SD is wide, for example 3987 ± 1366 for Tahoe in S1.



Queuing: weighted fair queuing protects the phone call

374 → 16 ms

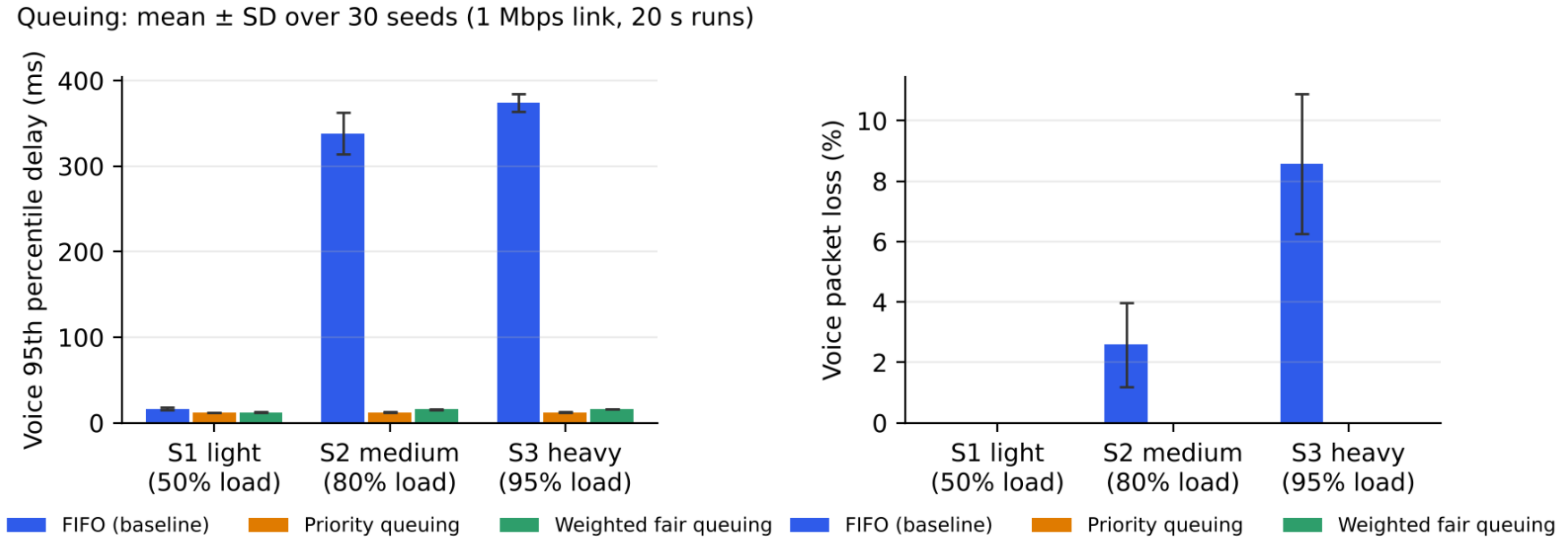
voice p95 delay at 95% data load (FIFO → WFQ)

8.57% → 0%

voice packet loss at 95% data load

12.6 vs 16.2

ms, priority vs WFQ: WFQ pays about 4 ms for fairness



78 tests check the labs against known answers

0xCBF43926

CRC-32 of the text 123456789, the standard check value

0xB861

IPv4 header checksum from the textbook example

0.184

Peak throughput of pure ALOHA, at G = 0.5

Same costs

Dijkstra and distance vector agree for every router pair

The other tests check invariants: every frame delivered exactly once and in order under random loss, WFQ never starves bulk traffic, and the same seed repeats exactly.

```
$ node tests/f1_team34_tests.js
78 passed, 0 failed
```

Analysis and limitations

What the results show

- Every improvement helps, and helps more under stress.
- Selective Repeat and WFQ differences are many SDs wide in S2 and S3.
- Protecting voice did not raise loss for bulk traffic.
- Priority is slightly faster for voice; WFQ avoids starvation.

What it does not show

- Simplified models: per-RTT TCP, fluid shared link, one queue port.
- Split horizon is tested on chains only, not meshes.
- No significance tests and no user study.
- Unit 7 (DNS, HTTP, FTP) and 802.11 CSMA/CA have no lab.

Conclusion and future work

Eight working labs in one zero-install page, four baseline-versus-improvement comparisons measured over 30 seeds, and 78 passing tests. The numbers can be reproduced with one command.

DNS lab

Recursive and iterative lookups with caching, to cover Unit 7

TCP handshake and UDP

Connection set-up and a comparison with UDP

CSMA/CA

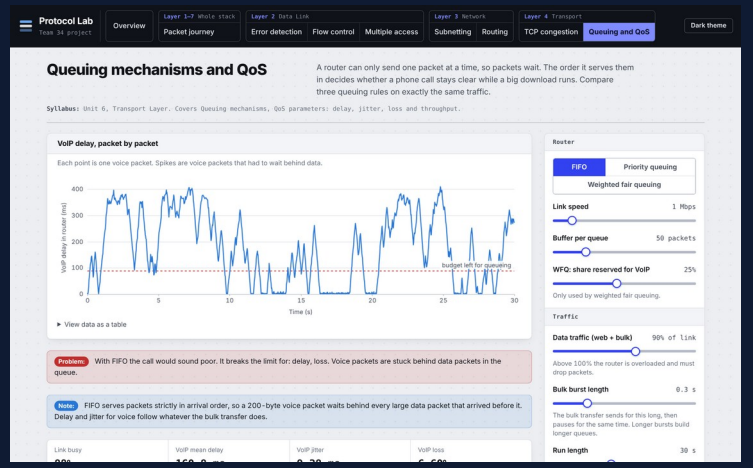
IEEE 802.11 medium access

Meshes and a classroom trial

Loops that split horizon cannot stop; measure learning with students

Thank you. Try it yourself

```
git clone https://github.com/savir18/CN.git
open f1_team34_index.html
node tests/f1_team34_tests.js
node scripts/f1_team34_experiments.js
```



Key references

Kurose and Ross, Computer Networking, 8th ed., 2021

Jacobson, Congestion avoidance and control, SIGCOMM CCR, 1988

Fall and Floyd, Tahoe, Reno and SACK TCP, SIGCOMM CCR, 1996

Demers, Keshav, Shenker, Fair queueing, SIGCOMM, 1989

Henderson, Lacage, Riley, ns-3, 2008; Lantz et al., Mininet, 2010